

Industrial Vision

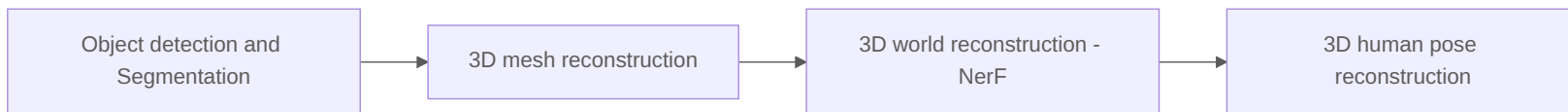
Lecture 1 - Low-level Vision & Image Filtering

Plan of the course

First half: classical Industrial Vision, comprising 50 to 60 per cent of the course



Second half: deep learning in Industrial Vision



Both halves are necessary. Deep networks do not solve every problem, and where they succeed, the classical formulation is usually what the network has learned implicitly.

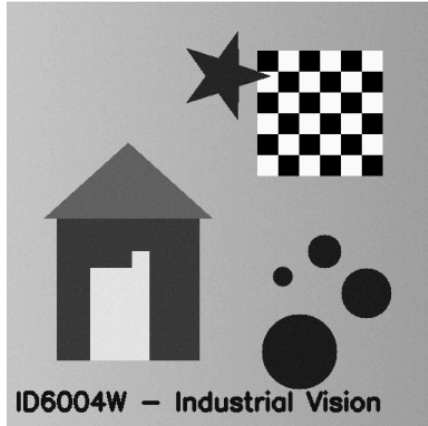
Outline of this lecture

1. We begin by asking what a simple 2D filter can achieve.
2. We define **filtering** on a pixel grid, in which every pixel is replaced by a weighted sum of its neighbors.
3. We compute two examples by hand, a **blur** and an **edge finder**.
4. We introduce the **Gaussian**, first on the continuous plane and then on the pixel grid.
5. We study **separability**, the condition under which a 2D filtering reduces to two 1D filterings.
6. We examine a live **demonstration**.
7. We derive **convolution** from the assumptions that define a **linear shift-invariant (LSI)** system.

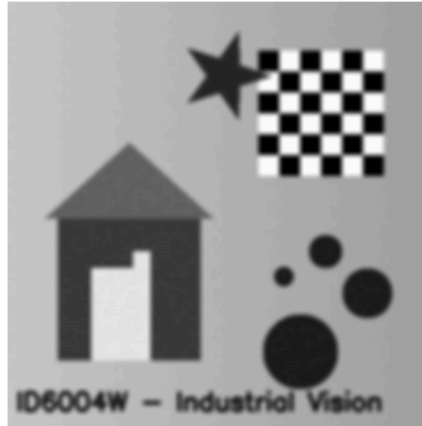
Low-level vision

Applications of a single 2D filter

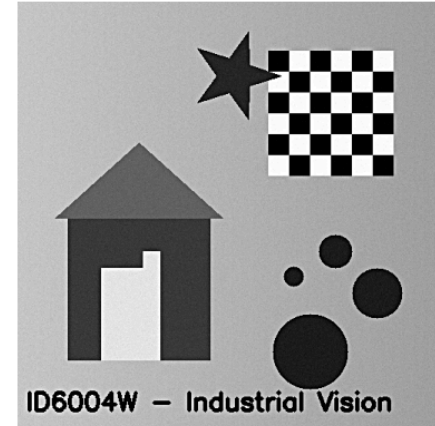
original



blur (Gaussian)



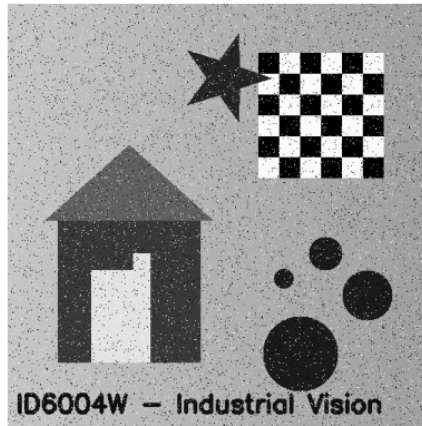
sharpen (unsharp mask)



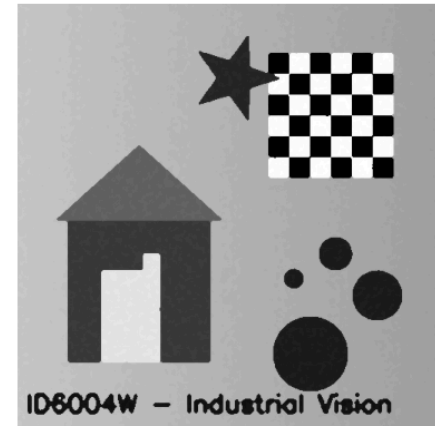
edges (|Sobel x|)



salt & pepper noise



median filtered



Tasks that extend beyond a single filter

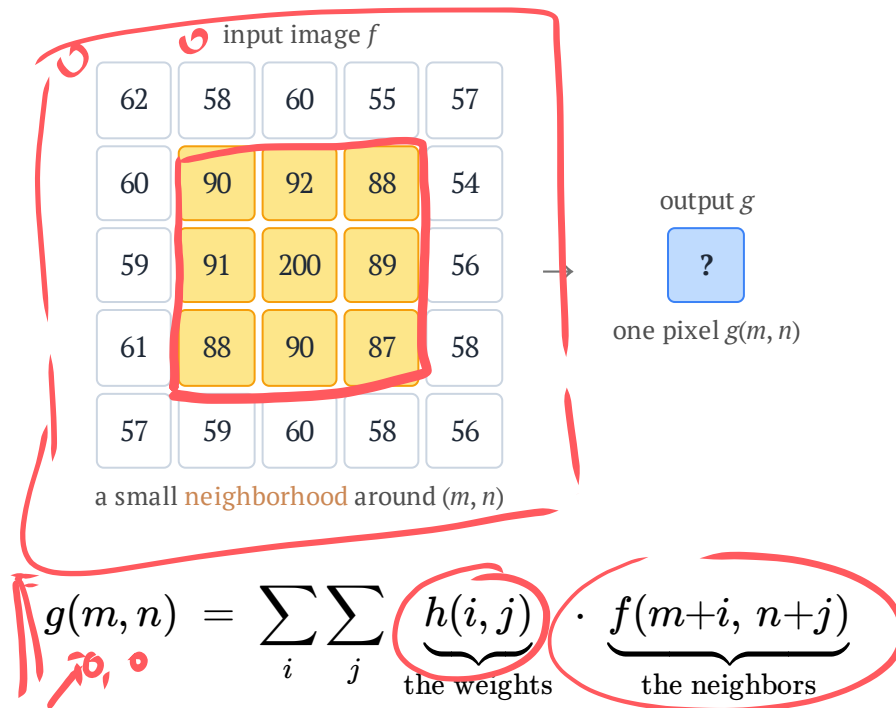
Task	Interpretation as filtering
De-blurring	Blurring is itself a filtering operation, so it can be undone by inverse filtering.
Super-resolution	The degradation is approximately an averaging followed by down-sampling. Recovery must add detail rather than interpolate it.
In-painting and out-painting	A local operator infers the content that a neighborhood ought to contain.
Template matching	A template is slid across the image and the response is large wherever it fits.

Filters also arise naturally, without anyone applying them deliberately:

- In defocus blur the optics of the lens act as a filter.
- In motion blur the relative motion smears the image, and this smearing is itself a filtering operation.

Definition of filtering

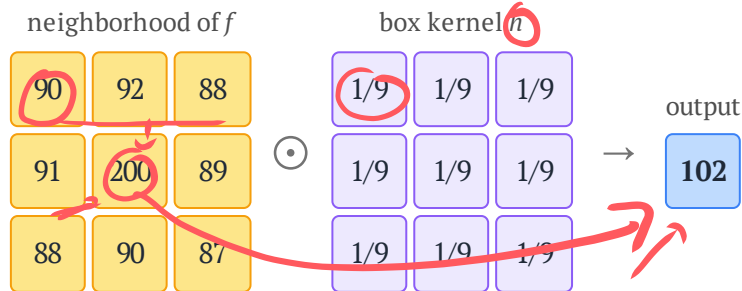
Every pixel is replaced by a **weighted sum of the pixels surrounding it**.



The small table of weights h is called the **kernel**, or equivalently the **filter**. The choice of weights determines the effect of the operation.

Example 1: a blur

Consider the **box** kernel, in which every weight equals $\frac{1}{9}$. It computes the plain **average** of the 3×3 neighborhood.



$$g = \frac{1}{9} (90 + 92 + 88 + 91 + 200 + 89 + 88 + 90 + 87) = \frac{915}{9} \approx 102$$

The bright value **200** is pulled down to **102** by its neighbors, so **the image is smoothed**.

Example 3: an edge finder

The machinery is unchanged and only the **weights** differ. The kernel now takes a **difference** across the image.

image: dark left, bright right					"vertical-edge" kernel h		
20	20	20	200	200	1	0	-1
20	20	20	200	200	1	0	-1
20	20	20	200	200	1	0	-1

Inside the dark region, where every value is 20:

$$(20+20+20) - (20+20+20) = 0$$

The region is flat, and so there is **no response**.

Straddling the boundary, with the left column at 20 and the right column at 200:

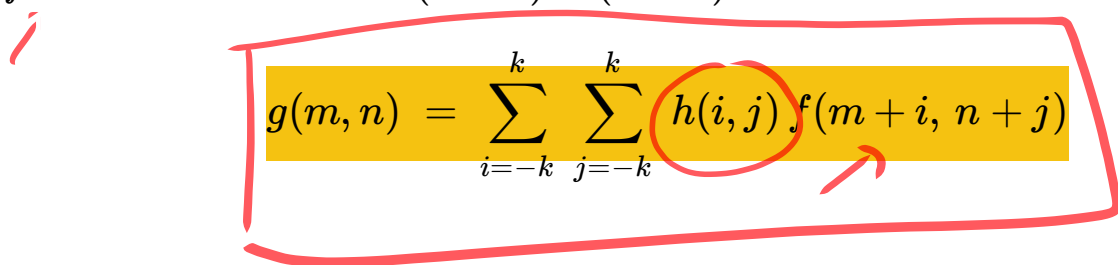
$$(20+20+20) - (200+200+200) = -540$$

The intensity changes, and so the response is **large**.

The filter **responds only where the intensity changes**, and such a location is called an **edge**. The box weights sum to **1** and produce an average, whereas the difference weights sum to **0** and indicate a change.

The operation is called convolution

Sliding a kernel over the whole image and accumulating the products at every position is **2D convolution**, written $g = f * h$. For a kernel of size $(2k+1) \times (2k+1)$:



The equation $g(m, n) = \sum_{i=-k}^k \sum_{j=-k}^k h(i, j) f(m + i, n + j)$ is displayed on a yellow background. A red bracket is positioned to the left of the equation. A red circle highlights the kernel term $h(i, j)$, and a red arrow points from this circle to the function term $f(m + i, n + j)$. The entire equation is enclosed within a larger red hand-drawn rectangular frame.

$$g(m, n) = \sum_{i=-k}^k \sum_{j=-k}^k h(i, j) f(m + i, n + j)$$

The operation is called convolution

$$g(m, n) = \sum_{i=-k}^k \sum_{j=-k}^k h(i, j) f(m + i, n + j)$$

The sum runs over the small support of the kernel, from $-k$ to k on each axis, which is why only a tiny neighborhood of each pixel is touched.

Boundaries. When the kernel is centered on a pixel at the image border, it **overhangs** the image by a margin of width k .

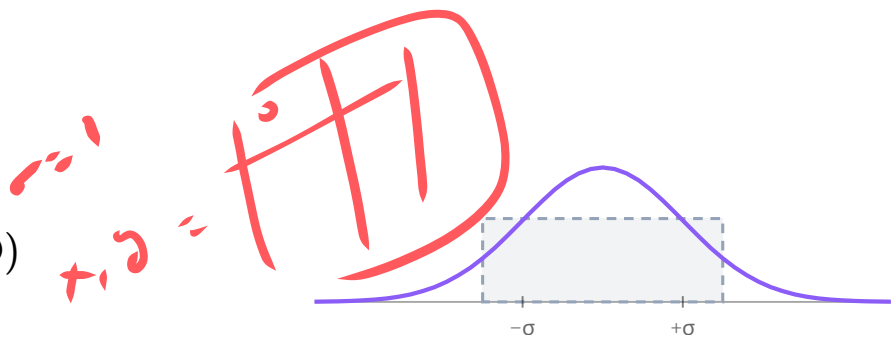
This is handled either by **padding** the image with zeros, replicated values or reflected values, or by **cropping** the output to the region in which the kernel fits entirely. Because h is small, only this narrow border is affected.

A better blur: the Gaussian

The box kernel weights every pixel of the window **equally**, so the far corner counts exactly as much as the immediate neighbor. It is more natural to let the weight **decay with distance**, and the function that does so is the **Gaussian**.

$$G_{\sigma}(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}} \quad (1D)$$

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (2D)$$



the Gaussian against the box, in one dimension

Here x and y are distances from the center pixel, measured in **pixels**, and σ is the width of the bell.

Unit area. $\int G_{\sigma} = 1$ and every weight is positive, so the filter is a genuine weighted average and the mean intensity is preserved.

Isotropy. The 2D form depends only on the radius $r^2 = x^2 + y^2$, so it blurs identically in every direction and favors no orientation.

One parameter. σ alone fixes the width of the bell, and therefore the amount of blur. It is the **scale** at which the image is being examined.

Constructing a discrete Gaussian: support and normalization

The Gaussian is defined on the continuous plane and is supported everywhere. To use it as a kernel we must give it a finite **support** and force it to **sum to 1**.

Support. Approximately 99% of the area of a Gaussian lies within $\pm 3\sigma$, so we keep 3σ pixels either side of the center:

$$\text{size} = \underbrace{3\sigma}_{\text{left}} + \underbrace{1}_{\text{center}} + \underbrace{3\sigma}_{\text{right}} = 2\lceil 3\sigma \rceil + 1$$

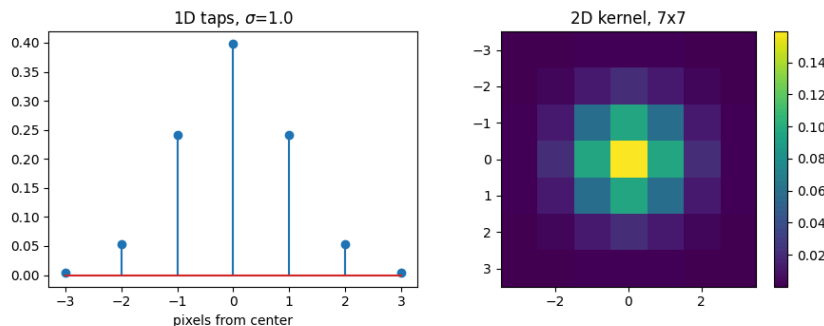
Written this way the size is **automatically odd**, so the center tap sits unambiguously on a pixel. For example, $\sigma=1$ gives $2(3)+1 = 7$ and $\sigma=2$ gives $2(6)+1 = 13$.

Normalization. We sample G_σ on the integer grid, $g(m, n) = \frac{1}{2\pi\sigma^2} e^{-\frac{m^2+n^2}{2\sigma^2}}$. Sampling and truncation destroy the unit area, so the samples are rescaled to restore it:

$$g'(m, n) = \frac{g(m, n)}{\sum_{m,n} g(m, n)}$$

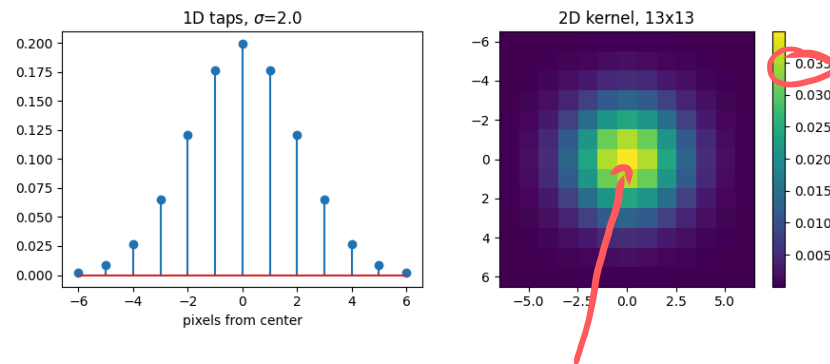
Two examples: $\sigma = 1$ and $\sigma = 2$

discrete Gaussian, $\sigma=1.0$



$3\sigma + 1 + 3\sigma = 7$, so a 7×7 kernel

discrete Gaussian, $\sigma=2.0$



$3\sigma + 1 + 3\sigma = 13$, so a 13×13 kernel

The shape is the same, only the width changes. Doubling σ doubles the number of taps and halves the peak, from 0.40 to 0.20, since the same unit of area is spread over more pixels.

Separability

Some 2D kernels **factor** into two 1D kernels, one for each axis:

$$h(x, y) = h_1(x) \cdot h_2(y)$$

A single 2D pass is then equivalent to **two inexpensive 1D passes**, in which the rows are filtered first and the columns afterwards.

The Gaussian of the previous two slides is the standard example:

$$h(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} = \underbrace{\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}}_{h_1(x)} \cdot \underbrace{\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{y^2}{2\sigma^2}}}_{h_2(y)}$$

The term $x^2 + y^2$ in the exponent separates into x^2 and y^2 , so that the kernel is the product of a horizontal blur and a vertical blur.

The computational cost of separable filtering

Let the image be of size $N \times N$ and the kernel of size $M \times M$.

Direct 2D convolution

- Each pixel requires M^2 multiplications.
- The image contains N^2 pixels.

$$\Rightarrow M^2 N^2 \text{ mults}$$

Separable filtering with two 1D passes

- The vertical pass requires M multiplications per pixel.
- The horizontal pass requires a further M multiplications per pixel.

$$\Rightarrow 2MN^2 \text{ mults}$$

$$\text{gain} = \frac{M^2 N^2}{2MN^2} = \frac{M}{2}$$

For a 7×7 Gaussian this amounts to **3.5 times fewer** multiplications. The count of additions does not alter the conclusion, since $2(M-1)N^2$ additions are required in place of $(M^2-1)N^2$. For large smoothing kernels this saving determines whether the operation is practical.

Factorization of the Sobel filter

$$\begin{bmatrix} 1 & 0 & -1 \\ \textcolor{red}{?} & 0 & -\textcolor{red}{?} \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ \textcolor{red}{?} \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

The two factors play distinct roles.

- The row $[1 \ 0 \ -1]$ takes a **difference across** the vertical edge and therefore detects the transition. It is exactly the edge finder of the previous example.
- The column $[1 \ 2 \ 1]^\top$ takes a center-weighted **average along** the edge and therefore suppresses noise.

Why average along and never across?

Averaging across the edge would smear the very structure you want; averaging along it leaves the edge intact while shrinking noise variance.

This pattern of **smoothing in one direction and differentiating in the other** recurs in the Harris detector, in SIFT and in the Lucas-Kanade method, and it is worth remembering.

The Gaussian on the pixel grid

The factorization given earlier was **continuous**. Sampling it on the pixel grid preserves it **exactly**, because a product sampled on a rectangular grid *is* an outer product:

$$H_{ij} = h(x_i, y_j) = h_1(x_i) h_2(y_j) \quad \implies \quad H = h_1 h_2^\top$$

The seven taps at $\sigma = 1$, that is $h_1 = [0.0044 \ 0.0540 \ 0.2420 \ 0.3991 \ 0.2420 \ 0.0540 \ 0.0044]$, generate the whole 7×7 kernel, whose entries are shown here in units of 10^{-4} :

$$\begin{bmatrix} 0.0044 \\ 0.0540 \\ 0.2420 \\ 0.3991 \\ 0.2420 \\ 0.0540 \\ 0.0044 \end{bmatrix} [0.0044 \ 0.0540 \ 0.2420 \ 0.3991 \ 0.2420 \ 0.0540 \ 0.0044] = 10^{-4} \begin{bmatrix} 0 & 2 & 11 & 18 & 11 & 2 & 0 \\ 2 & 29 & 131 & 216 & 131 & 29 & 2 \\ 11 & 131 & 586 & 966 & 586 & 131 & 11 \\ 18 & 216 & 966 & 1592 & 966 & 216 & 18 \\ 11 & 131 & 586 & 966 & 586 & 131 & 11 \\ 2 & 29 & 131 & 216 & 131 & 29 & 2 \\ 0 & 2 & 11 & 18 & 11 & 2 & 0 \end{bmatrix}$$

The integer approximation used in practice behaves in the same way, and its 1D factor is the familiar one from the Sobel filter:

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \frac{1}{4} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot \frac{1}{4} [1 \ 2 \ 1]$$

Forty-nine entries are determined by seven numbers. The kernel is of rank one, and the second singular value is 10^{-17} relative to the first, which is machine precision rather than approximation. The corner entry is 1.9×10^{-5} and only *displays* as zero, which is precisely why truncating at $\pm 3\sigma$ costs nothing.

Separability is a statement about rank

Written as a matrix, the condition $h(x, y) = h_1(x) h_2(y)$ is an **outer product**, and a nonzero matrix is an outer product of two vectors precisely when its rank is one:

$$H = h_1 h_2^\top \iff \boxed{\text{rank}(H) = 1}$$

Separable, of rank one

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

The sampled Gaussian is of rank one **exactly**, since sampling a product on a rectangular grid returns an outer product.

Of rank two, and therefore not separable

$$\nabla^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

The first and third rows coincide, so the rank is at most two, and the first two rows are independent, so it is exactly two.

Rank deficiency alone is therefore **not** sufficient. A 5×5 kernel may have rank 2, 3 or 4, and is separable in none of these cases. Separability is the single value $r = 1$.

A catalogue of common filters

The **box**, or uniform, filter computes a plain average.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Its entries **sum to 1**. A passive system redistributes energy but cannot introduce it, just as a lens cannot add light, and therefore the mean intensity is preserved.

The **gradient**, or difference, filters are given below.

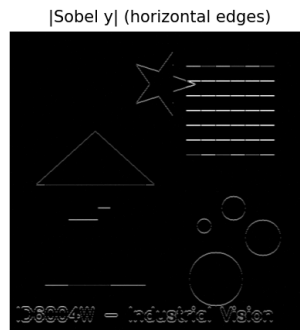
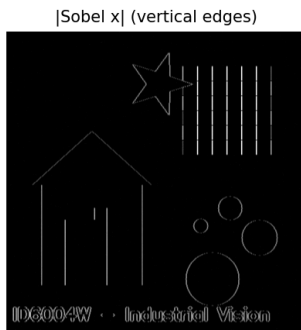
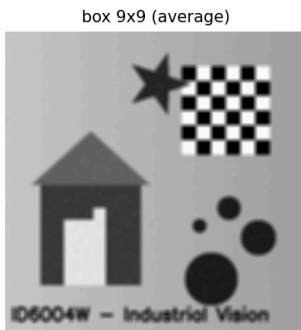
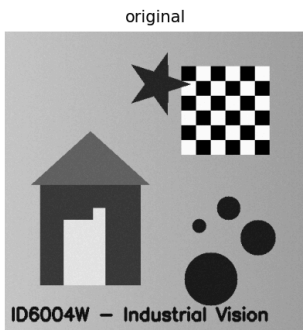
$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

vertical edges

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

horizontal edges

They respond wherever the intensity changes and give zero on constant regions, because their coefficients **sum to 0**.



Demonstration

image *ideal step edges, exact corners — the textbook case*

Original

Box 3×3

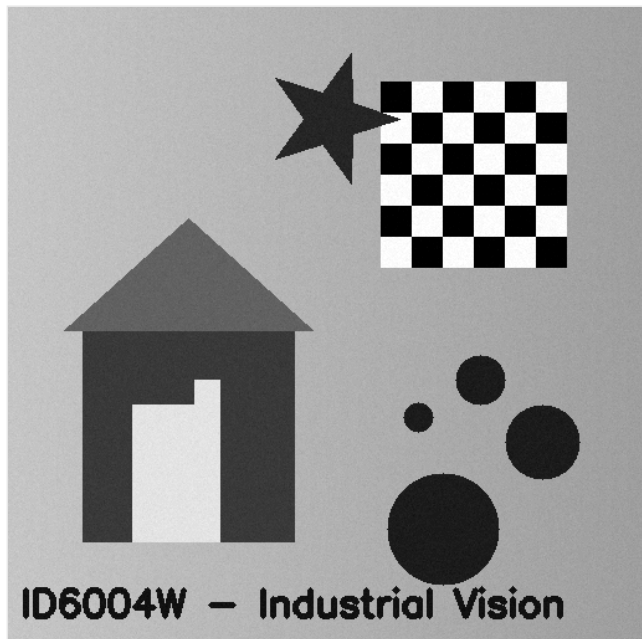
Box 9×9

Gaussian $\sigma=3$

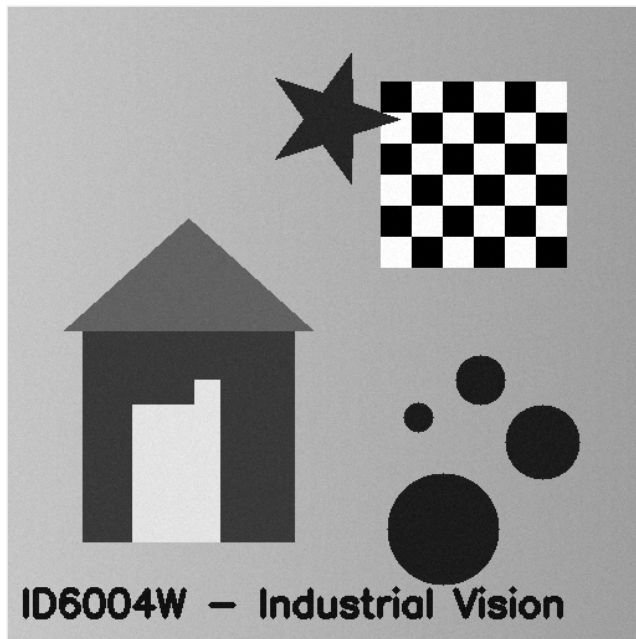
Sobel x

Sobel y

Sharpen



$f(m, n)$

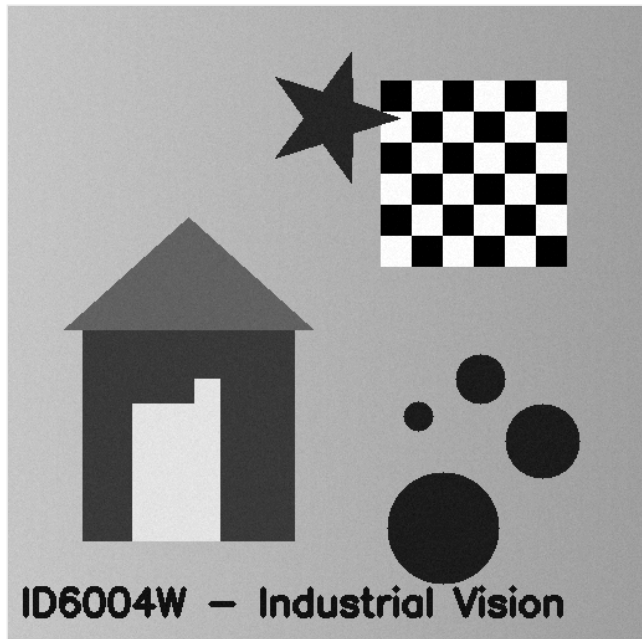


$g(m, n) = (f * h)(m, n)$ — Original

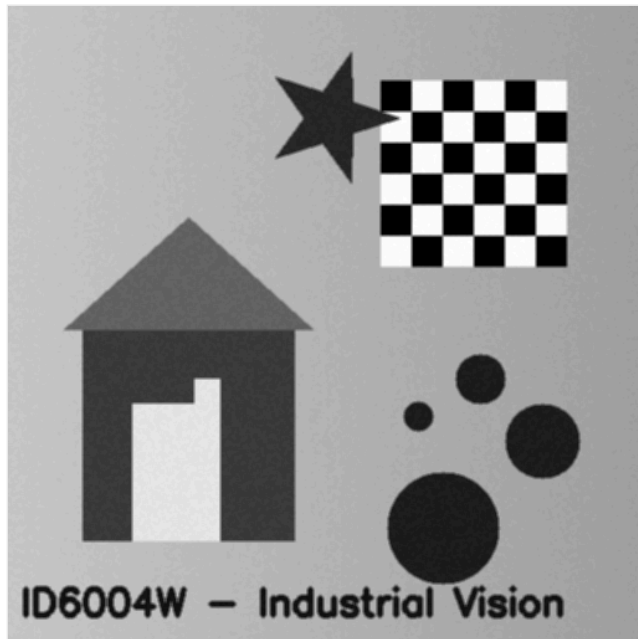
Demonstration — design your own kernel

image Synthetic scene ▼ *ideal step edges, exact corners — the textbook case*

presets: Box blur Weighted blur Identity Sharpen Sobel x Sobel y ☒ normalize ($\div \sum \text{weights} = 9$)



$f(m, n)$



$g(m, n) = (f * h)(m, n)$

custom kernel h — edit the weights

1	1	1
1	1	1
1	1	1

weights summing to 1 blur · to 0 find edges

Type any nine weights into the kernel and watch the result update. Keep **normalize** on to divide by the sum of the weights, so a blur preserves the mean intensity; turn it off to see raw responses (useful for the difference kernels).

Why this particular operation?

So far we have slid kernels across the image and accumulated products.

We now establish that **convolution is the only operation** consistent with **two reasonable assumptions**.

Linearity together with shift invariance implies convolution. The operation is therefore **derived** rather than postulated.

The two assumptions of an LSI system

Assumption 1, linearity. A system $\mathcal{T}$ is linear if **superposition** holds:

$$\begin{aligned} f_1 \mapsto g_1, f_2 \mapsto g_2 &\Rightarrow f_1 + f_2 \mapsto g_1 + g_2 \\ &\Rightarrow \alpha f_1 \mapsto \alpha g_1 \quad (\alpha \in \mathbb{R} \text{ or } \mathbb{C}) \end{aligned}$$

Assumption 2, shift invariance in space. The operator behaves identically at every location in the image:

$$f(m, n) \mapsto g(m, n) \quad \Rightarrow \quad f(m - m_0, n - n_0) \mapsto g(m - m_0, n - n_0)$$

A system that satisfies both assumptions is called a **linear shift-invariant (LSI)** system. Informally, the filter is the same at every position in the image.

The 2D impulse as a building block

The **Kronecker delta** represents a single illuminated pixel at the origin:

$$\delta(m, n) = \begin{cases} 1 & m = 0, n = 0 \\ 0 & \text{else} \end{cases}$$

Any image can be written as a sum of scaled and shifted impulses. This is the **sifting property**:

$$f(m, n) = \sum_{m'} \sum_{n'} f(m', n') \delta(m - m', n - n')$$

Each pixel value $f(m', n')$ scales the impulse located at (m', n') .

If a single impulse is applied to the system, the resulting output is called the **impulse response** h :

$$\delta(m, n) \mapsto h(m, n; 0, 0)$$

$$\delta(m - m', n - n') \mapsto h(m, n; n', n')$$

Derivation of convolution from the two assumptions

$$f(m, n) = \sum_{m'} \sum_{n'} f(m', n') \delta(m - m', n - n')$$

Applying linearity, the response to a weighted sum of impulses is the weighted sum of the individual responses:

$$g(m, n) = \sum_{m'} \sum_{n'} f(m', n') h(m, n; m', n')$$

Applying shift invariance, a shifted impulse produces the same response, merely shifted:

$$h(m, n; m', n') = h(m - m', n - n'; 0, 0)$$

$$g(m, n) = \sum_{m'} \sum_{n'} f(m', n') h(m - m', n - n') = \sum_{m'} \sum_{n'} h(m', n') f(m - m', n - n')$$

This is **2D convolution**, written $g = f * h$, and it is precisely the operation that was slid across the grid earlier. It is imposed by the two LSI assumptions rather than chosen.

A note on the sign. The earlier slides wrote $f(m+i, n+j)$, which is **correlation**; the derivation gives $f(m-m', n-n')$, which is **convolution**. The two differ by a flip of the kernel: invisible for a symmetric kernel such as the box or the Gaussian, and a change of sign for an antisymmetric one such as $\begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$. Deep learning libraries implement correlation and call it convolution for exactly this reason.

Summary

- **Filtering** replaces each pixel by a **weighted sum of its neighbors**. The small table of weights is the **kernel**, and the choice of weights determines the effect.
- The same operation with different weights produces different effects. A **box** kernel, whose weights sum to one, averages the neighborhood and blurs the image. A **difference** kernel, whose weights sum to zero, responds only where the intensity changes.
- Sliding the kernel and accumulating the products is **convolution**, $g = f * h$, with the sum taken over the small kernel support. The image boundary requires separate treatment.
- The **Gaussian** replaces the plain average by a weighted one, with the weight decaying with distance. It has unit area, is isotropic and is controlled by the single parameter σ . On the grid it is truncated to $2\lceil 3\sigma \rceil + 1$ taps and renormalized.
- **Separable** kernels, for which $h = h_1 h_2$, such as the Gaussian and the Sobel kernels, reduce the cost from $M^2 N^2$ to $2MN^2$, a gain of a factor $M/2$.
- Convolution is not an arbitrary choice. **Linearity and shift invariance** together imply convolution, so the operation is derived rather than postulated.

References

1. R. Szeliski, *Industrial Vision: Algorithms and Applications*, 2nd ed. Springer, 2022. See §3.2 on linear filtering and separability.
2. D. A. Forsyth and J. Ponce, *Industrial Vision: A Modern Approach*, 2nd ed. Pearson, 2012. See **Ch. 4, "Linear Filters"**, in particular §4.1 on convolution, §4.2 Shift invariant Linear Systems and §4.5 on filters as templates.

Demo code: `lec01_filtering.py` (posted on the LMS portal)